

Docker uses an engine-client-server model, wherein the Docker Engine is responsible for creating and running containers. A Docker CLI can be used to issue Docker commands from a command-line terminal on a given platform, and Docker Hub or other registries store and share available container images.

What is a Dockerfile?

A Dockerfile is a plain text file that contains step-by-step instructions or commands to assemble a Docker image. Think of it as a recipe that tells Docker exactly how to build your image, layer by layer.

Using a Dockerfile means you can automate the image creation process, making it consistent, repeatable, and easy to share with others. Here's an example of a Dockerfile.

```
Use official Python image as base
FROM python:3.10
Set working directory
WORKDIR /app
```

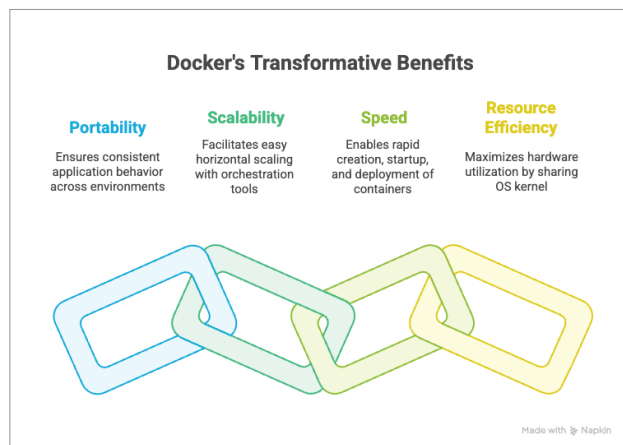


Figure 2: Benefits of using Docker

Key use cases of Docker

Use case	Description	How Docker helps	Example
DevOps	Integrating development and operations for faster, reliable software delivery	Consistent environments across dev, test, and production; automates deployment; simplifies rollback and updates	Using Docker containers to package apps and automate deployment via tools like Jenkins, GitLab CI, or CircleCI
Microservices	Architecting applications as a collection of loosely coupled, independently deployable services	Isolates each microservice in its own container; simplifies scaling individual services; enables polyglot environments	Running separate containers for authentication, payments, and user profiles that can be developed and scaled independently
CI/CD pipelines	Automating code integration, testing, and deployment to deliver software rapidly and reliably	Provides consistent test environments; speeds up build and test phases with reusable container images; supports parallel testing	Running automated tests inside containers during each commit, then deploying containerised apps seamlessly

```
Copy current directory contents to /app inside container
COPY . .
Install Python dependencies
RUN pip install -r requirements.txt
Make port 80 available to the world outside container
EXPOSE 80
Define environment variable
ENV ENVIRONMENT=production
Run the app
CMD ["python", "app.py"]
```

Getting started with Docker

For basic installation in Windows and Mac, download and install 'Docker Desktop' from docker.com. For Linux, you can use your package manager. For example, on Ubuntu:

```
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

The first container run is:

```
docker run hello-world
```

Building and Running Containers

```
# Use official Python runtime as base
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Copy current directory contents into container at /app
COPY . .

# Install dependencies
RUN pip install -r requirements.txt
```